



RAPPORT DE DÉVELOPPEMENT DE MODULE

Création et Analyse du Module Hello World sous
Drupal

Réalisé par
YAHYA EL OMARI

Encadrants
ABDELMAJID BENDRIF
HAMZA BAHLAOUANE

3 mars 2026

Table des matières

Introduction	2
1 Mise en Place du Module Hello World	2
1.1 Création de la Route et du Contrôleur	2
1.2 Création d'un Bloc Personnalisé	3
2 Expérimentations et Analyse des Configurations	4
2.1 Désinstallation et Réinstallation du Module	4
2.2 Modification de "core_version_requirement"	4
2.3 Ajout d'une Dépendance	5
2.4 Ajout d'un Fichier composer.json	5
2.5 Déplacement ou Suppression des Fichiers du Module	6
3 Gestion des Liens et de la Navigation	7
3.1 Menu items (Liens de menu)	7
3.2 Action links (Liens d'action)	7
3.3 Local task links (Onglets / Tâches locales)	8
4 Intégration d'un Sous-Système Complet (HTTP, Twig, Assets, i18n)	8
4.1 Création du Service API	8
4.2 Injection de Dépendances dans le Contrôleur	9
4.3 Définition du Template Twig et Traduction (i18n)	9
4.4 Attachement d'Assets (CSS/JS)	9
5 Questions Techniques (FAQ)	10

INTRODUCTION

Ce rapport détaille la création et la configuration d'un module personnalisé nommé "Hello World" sous Drupal. Il illustre la mise en place d'un contrôleur pour une page personnalisée, la création d'un bloc, ainsi que l'analyse des différentes configurations et erreurs courantes lors de la gestion des modules.

1 Mise en Place du Module Hello World

1.1 Création de la Route et du Contrôleur

La première étape de la création du module a consisté à déclarer le module via son fichier `hello_world.info.yml`. En parallèle, nous avons défini une route dans `hello_world.routing.yml` pointant vers un contrôleur personnalisé.

Listing 1 – Aperçu du contrôleur HelloController.php

```
<?php

namespace Drupal\hello_world\Controller;

use Drupal\Core\Controller\ControllerBase;

class HelloController extends ControllerBase {
    public function content() {
        return [
            '#type' => 'markup',
            '#markup' => $this->t('Hello World'),
        ];
    }
}
```

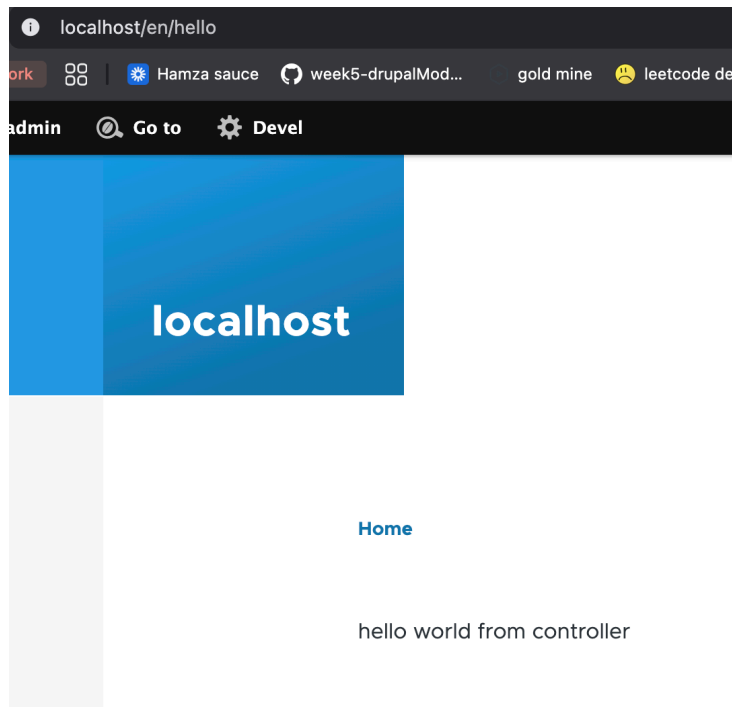


FIGURE 1 – Affichage de la page "Hello World" générée par le contrôleur personnalisé.

1.2 Création d'un Bloc Personnalisé

Un bloc personnalisé a été développé en créant un plugin. Une fois le code en place et les caches vidés, ce bloc est devenu disponible dans l'interface d'administration de Drupal, prêt à être placé dans n'importe quelle région du thème.

Listing 2 – Aperçu du plugin HelloBlock.php

```
<?php

namespace Drupal\hello_world\Plugin\Block;

use Drupal\Core\Block\BlockBase;

/**
 * Provides a 'Hello' Block.
 *
 * @Block(
 *   id = "hello_block",
 *   admin_label = @Translation("Hello block"),
 * )
 */
class HelloBlock extends BlockBase {
  public function build() {
    return [
      '#markup' => $this->t('Hello, World!'),
    ];
  }
}
```

```
}
```

Hello block

Hello, World from block!

RSVP Form

Name *

Email *

Submit

FIGURE 2 – Le bloc "Hello World" affiché sur l'interface du site.

2 Expérimentations et Analyse des Configurations

Afin de mieux comprendre le fonctionnement de Drupal et la gestion de ses dépendances, plusieurs manipulations et tests ont été effectués sur le module.

2.1 Désinstallation et Réinstallation du Module

Question : *Désinstaller les modules, puis les installer à nouveau, que se passe-t-il ?*

En désinstallant le module, Drupal supprime toutes les configurations actives liées à celui-ci (comme le placement du bloc "Hello block" dans les régions du thème). Le code reste présent sur le serveur, mais Drupal n'en tient plus compte. Lors de la réinstallation, le module repart d'un état vierge : il faut reconfigurer le placement des blocs et potentiellement réinstaller ses entités de configuration si elles n'existent pas déjà.

2.2 Modification de "core_version_requirement"

Question : *Modifier core_version_requirement vers une version inférieure, que se passe-t-il ?*

Si l'on définit `core_version_requirement` sur une version incompatible (par exemple `^8` ou `^9` alors que le site tourne sous Drupal 10/11), Drupal n'empêche pas fatalement l'activation du module, mais il affiche un avertissement explicite. Ce message permet à l'administrateur de savoir que bien que le module s'installe, il n'est potentiellement pas supporté par la version installée du cœur de Drupal (et des instabilités pourraient survenir).



FIGURE 3 – Avertissement de Drupal suite à la restriction de version non supportée.

2.3 Ajout d'une Dépendance

Question : *Ajouter le module "workflows" comme dépendance, que se passe-t-il ?*

Dans le fichier `hello_world.info.yml`, l'ajout de la ligne `- drupal:workflows` dans la section `dependencies` oblige Drupal à vérifier la présence de ce module. Si `workflows` n'est pas activé, l'installation de `hello_world` échouera avec un message d'erreur mentionnant la dépendance manquante ("Unable to install modules: module 'hello_world' is missing its dependency module workflows"). Une fois la dépendance satisfaite (module workflows activé), notre module pourra s'installer.

2.4 Ajout d'un Fichier composer.json

Question : *Ajouter un fichier composer.json à vos modules.*

Conformément à la documentation officielle, un fichier `composer.json` a été ajouté à la racine du module. Ce fichier permet de déclarer des dépendances vers des bibliothèques externes et de définir la déclaration PSR-4 pour le namespace de notre module (`Drupal\hello_world\`). C'est essentiel pour standardiser le module si l'on souhaite le publier sur Drupal.org.

```

new_drupal > web > modules > custom > hello_world > {} composer.json > {} require
1  {
2      "name": "mac/hello_world",
3      "description": "hello world module",
4      "type": "drupal-module",
5      "license": "GPL-2.0",
6      "autoload": {
7          "psr-4": {
8              "Mac\\HelloWorld\\": "src/"
9          }
10     },
11     "authors": [
12         {
13             "name": "yahyaalomari",
14             "email": "y.elomari@void.fr"
15         }
16     ],
17     "require": {}
18 }

```

FIGURE 4 – Configuration du fichier composer.json pour le module.

2.5 Déplacement ou Suppression des Fichiers du Module

Question : *Supprimer ou déplacer vos modules vers un autre dossier, que se passe-t-il ? Quel message apparaît dans Drupal ? Que disent les logs ('drush ws') ?*

Si l'on supprime ou déplace physiquement le dossier d'un module alors que celui-ci est toujours activé dans Drupal, le CMS ne parvient plus à charger les ressources qu'il garde en mémoire.

Avant l'action, le système est stable et tout fonctionne :

ID	Date	Type	Severity	Message
454	03/Mar 11:48	locale	Notice	The configuration was successfully updated. 182 configuration objects updated.
455	03/Mar 11:48	system	Info	hello_world module installed.
452	03/Mar 11:21	locale	Notice	The configuration was successfully updated. 182 configuration objects updated.
451	03/Mar 11:21	system	Info	workflows module installed.
450	03/Mar 11:14	system	Info	hello_world module uninstalled.
449	03/Mar 10:41	locale	Notice	The configuration was successfully updated. 182 configuration objects updated.

FIGURE 5 – État normal du site avant suppression ou déplacement du module.

Une fois les fichiers supprimés, Drupal tente d'utiliser son Service Container et ses définitions pour charger les classes du module (comme le contrôleur ou le plugin de bloc). Sans trouver les fichiers PHP via le système d'autoload, une erreur fatale survient : l'injection de dépendances échoue. Le site affiche à l'utilisateur un message d'erreur générique : "The website encountered an unexpected error" ou mentionne une "Dependency injection failed".

```

Stack trace:
#0 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(426): Drupal\Component\Depen
dependencyInjectionContainer->get('module_handler', 1)
#1 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(237): Drupal\Component\Depen
dependencyInjectionContainer->resolveServicesAndParameters(Array)
#2 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(177): Drupal\Component\Depen
dependencyInjectionContainer->createService(Array, 'config.type')
#3 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(426): Drupal\Component\Depen
dependencyInjectionContainer->get('config.type', 1)
#4 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(237): Drupal\Component\Depen

```

FIGURE 6 – Erreur fatale de type "Dependency Injection Failed" affichée.

Puisque l'interface web peut planter globalement, l'outil en ligne de commande Drush devient précieux. En utilisant la commande **drush ws** (Watchdog Show), on peut interroger les journaux système de Drupal. Les logs révèlent la

nature exacte du blocage, indiquant par exemple que le Service ou la Définition de routing pour ce module spécifique n'a pas pu être trouvée, créant une exception fatale non interceptée.

```
+ new drupal ./vendor/bin/drush ws
Failed to log error: AssertionError: The file specified by the given app root, relative path and file name (/Users/mac/Documents/drupal/new_drupal/web/modules/contrib/asset_injector/asset_injector.info.yml) do not exist. in assert() (line 75 of /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Core/Extension/Extension.php). #0 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Core/Extension/Extension.php(75): assert(false, 'The file speci...')
#1 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Core/Extension/ModuleHandler.php(133): Drupal\Core\Extension\Extension->construct('/Users/mac/Docu...', 'module', 'modules/contrib...', 'asset_injector...')
#2 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(259): Drupal\Core\Extension\ModuleHandler->construct('/Users/mac/Docu...', Array, Object(Drupal\Core\KeyValueStore\KeyValueFactory), Object(Drupal\Core\Utility\CatLibResolver), Object(Drupal\Core\Cache\DefaultBackend))
#3 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Component/DependencyInjection/Container.php(177): Drupal\Component\DependencyInjection\Container->createService(Array, 'module_handler')
#4 /Users/mac/Documents/drupal/new_drupal/web/core/lib/Drupal/Core/DrupalKernel.php(613): Drupal\Component\DependencyInjection\Container->get('module_handler')
#5 /Users/mac/Documents/drupal/new_drupal/vendor/drush/drush/src/Boot/DrupalBoot8.php(286): Drupal\Core\DrupalKernel->preHandle(Object(Symfony\Component\HttpFoundation\Request))
```

FIGURE 7 – Trace de l'erreur fatale inspectée via la commande drush ws.

3 Gestion des Liens et de la Navigation

Pour intégrer un module de manière fluide dans l'administration et l'expérience utilisateur de Drupal, il est courant de définir différents types de liens. Ces définitions se font via des fichiers YAML spécifiques.

3.1 Menu items (Liens de menu)

Les éléments de menu sont des entrées individuelles utilisées pour la navigation du site. Dans notre module, l'ajout d'une entrée dans le menu d'administration se fait via le fichier **hello_world.links.menu.yml**.

Listing 3 – hello_world.links.menu.yml

```
hello_world.admin:
  title: "Hello module settings"
  description: "example of how to make an admin settings page link"
  parent: system.admin_config_development
  route_name: hello_world.content
  weight: 100
```

3.2 Action links (Liens d'action)

Les liens d'action servent aux opérations principales sur une page, comme un bouton "Ajouter du contenu". Ils mènent souvent à des formulaires spécifiques d'un contexte. Ils sont définis dans **hello_world.links.action.yml**.

Listing 4 – hello_world.links.action.yml

```
hello_world.action_hello:
  title: "Say Hello!"
  route_name: hello_world.content
  appears_on:
    - system.admin_config_development
  weight: 0
```


3.3 Local task links (Onglets / Tâches locales)

Les "local tasks" s'affichent généralement sous forme d'onglets, permettant un accès rapide aux tâches connexes (comme l'affichage, l'édition ou la suppression d'un contenu). Ils se configurent dans `hello_world.links.task.yml`.

Listing 5 – `hello_world.links.task.yml`

```
hello_world.settings_tab:
  title: 'Settings'
  route_name: hello_world.content
  base_route: system.admin_config_development
```

4 Intégration d'un Sous-Système Complet (HTTP, Twig, Assets, i18n)

Afin d'étendre notre module `hello_world`, nous avons implémenté la consommation d'une API externe via le client HTTP Guzzle, géré les exceptions avec le Logger de Drupal, organisé le rendu avec Twig et embelli le résultat avec des Assets (CSS/JS). Le tout a été traduit nativement grâce à l'API d'internationalisation (i18n).

4.1 Création du Service API

Au lieu de charger la logique métier dans le contrôleur, nous avons créé un service dédié. Le client HTTP natif de Drupal (qui implémente `\GuzzleHttp\ClientInterface`) et le `logger.factory` sont injectés dans ce service via `hello_world.services.yml`.

Listing 6 – Aperçu de `HelloApiClient.php`

```
class HelloApiClient {
    use StringTranslationTrait;
    //... constructeur avec ClientInterface et
    //... LoggerChannelFactoryInterface

    public function fetchData() {
        try {
            $response = $this->httpClient->request('GET', 'https://
                jsonplaceholder.typicode.com/todos/1');
            //... traitement des donnees
        } catch (\Exception $e) {
            $this->logger->error('API_request_failed: @message', [
                '@message' => $e->getMessage()]);
            $result['error'] = $this->t('Could not fetch data from the
                remote server at this time. ');
        }
        return $result;
    }
}
```

4.2 Injection de Dépendances dans le Contrôleur

Notre **HelloController** utilise **ContainerInjectionInterface** pour charger automatiquement notre nouveau service **hello_world.api_client**. Il permet aussi de renvoyer un "Render Array" pointant vers un template Twig personnalisé au lieu d'une simple chaîne de balisage, facilitant la maintenabilité.

4.3 Définition du Template Twig et Traduction (i18n)

Via le hook **hook_theme()**, nous définissons comment Drupal doit passer les variables (**data** et **error**) à notre template **hello-world-remote.html.twig**. Afin de supporter l'internationalisation, toutes les chaînes de caractères statiques du template sont filtrées à travers **|t**, permettant à Drupal de repérer ces chaînes dans l'interface de traduction.

4.4 Attachement d'Assets (CSS/JS)

Pour styliser la donnée, nous définissons une librairie d'assets dans **hello_world.libraries** (nommée **remote_styles**), qui dépend de composantes cœurs comme **core/drupal** et **core/once**.

Listing 7 – Aperçu de **hello-world-remote.html.twig** avec intégration CSS/JS

```
{{ attach_library('hello_world/remote_styles') }}
<div class="hello-world-remote-wrapper">
  <h2>{{ 'Remote API Data'|t }}</h2>
  <button id="hello-world-toggle" class="hello-world-toggle-btn">{{
    'Toggle Styles'|t }}</button>
  <!-- ... conditionnalite if/else ... -->
</div>
```

Un script "**hello-world-remote.js**" modifie dynamiquement le DOM en basculant une classe **toggled-style** via l'API **Drupal.behaviors** et la méthode **once**, garantissant que l'auditeur d'évènements n'est attaché qu'une seule fois au chargement.

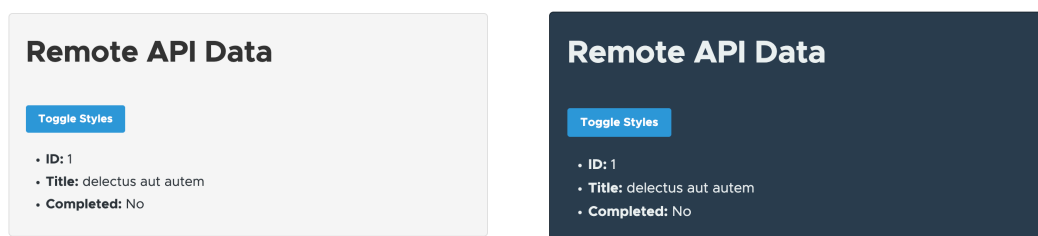


FIGURE 8 – À gauche : L'affichage standard avec CSS. À droite : L'interface modifiée après le basculement piloté par la logique JavaScript.

5 Questions Techniques (FAQ)

1. Comment contrôler ou trier les menus (weight)?

Dans le fichier `links.menu.yml`, on rajoute une ligne `weight`. Si on met un chiffre plus petit comme -10, ça monte dans la liste du menu, et un plus grand ça descend.

2. Comment configurer des menus enfants?

On utilise la clé `parent` dans notre fichier yml. Par exemple, si je veux le mettre sous la section de développement, je mets `parent: system.admin_config_development`.

3. Comment récupérer une chaîne de requête (query string) dans un contrôleur?

On doit ajouter l'objet `Request` de Symfony dans les paramètres de notre fonction. Après, on peut juste faire `$request->query->get('mon_param')` pour récupérer la valeur de l'URL.

4. Que sont Guzzle et Logger?

Guzzle c'est une librairie PHP intégrée dans Drupal qui permet de faire des requêtes HTTP (genre appeler une API externe facilement). Logger c'est un service pour écrire des messages d'erreur ou d'info dans les logs du site pour qu'on puisse débbuger quand un truc plante.

5. Utilisez Logger pour enregistrer un message dans votre contrôleur, où ces messages apparaissent-ils? Et comment les consulter?

On peut faire `$this->getLogger('mon_module')->info('mon message')`. Ces messages vont s'afficher dans l'interface admin de Drupal sous la page "Rapports > Messages récents du journal". On peut aussi les voir plus vite dans le terminal avec la commande `drush ws`.

6. Où trouver les services définis par le cœur de Drupal (un fichier YAML)? Ils sont tous listés dans le gros fichier `core/core.services.yml`.

7. Comment injecter d'autres services nécessaires dans votre service personnalisé?

Dans notre fichier `services.yml`, on rajoute une ligne `arguments: ['@nom_du_service']` sous notre classe. Ensuite on les récupère dans les paramètres de la fonction `__construct()` de notre classe PHP.

8. Comment retourner un modèle Twig dans un contrôleur?

Au lieu de renvoyer du HTML direct, on renvoie un tableau de rendu avec la clé `'#theme' => 'nom_de_mon_theme'`. Mais avant, il a fallu déclarer ce nom dans une fonction `hook_theme` dans notre fichier `.module`.

9. Comment ajouter le JS externe [tailwindcss](#) à votre contrôleur?

D'abord, on le déclare dans notre fichier `libraries.yml` avec `type: external` et le lien URL. Ensuite, dans notre contrôleur, on attache cette librairie à notre affichage avec `['#attached' => ['library' => ['hello_world/nom_de_la_lib']]]`.

10. Étant donné `$this->t('Hello, @name!', ['@name' => $username])`, quel est le mot de recherche que vous utiliserez pour le chercher dans `/admin/config/regional/translate`?

Il faut chercher la chaîne exacte sans les variables remplacées, donc je cherche "Hello, @name!".

11. Comment rendre une chaîne traduisible dans les fichiers javascript de Drupal?

On utilise `Drupal.t('Mon texte')` dans le code JS. Il faut juste s'assurer d'avoir bien mis `core/drupal` dans les dépendances de notre librairie yaml.

12. Utilisez `drush php` pour obtenir le service `path.alias.manager`. Je lance d'abord `drush php` dans mon terminal. Ensuite je tape `$alias_manager = \Drupal::service('path_alias.manager');` (Dans les nouvelles versions de Drupal 10+, c'est `path_alias.manager` au lieu de `path.alias.manager`).

13. Votre travail est d'obtenir l'alias d'un chemin comme `node/1`, utilisez votre service pour le récupérer.

Toujours depuis `drush php`, je fais `echo $alias_manager->getAliasByPath('/node/1');` et ça m'affiche l'URL sympa formatée de la page.

14. Utilisez `Link` et `Url` pour obtenir l'URL complète d'une de vos routes.

On utilise la classe `Core` de Drupal comme ça :

```
$url = \Drupal\Core\Url::fromRoute('ma_route', [], ['absolute' => TRUE])->toString();
```

15. Comment envoyez-vous une réponse JSON structurée dans un contrôleur plutôt qu'un affichage de page standard?

Au lieu de renvoyer le tableau de rendu normal, on instancie directement une classe `JsonResponse` de Symfony :

```
return new \Symfony\Component\HttpFoundation\JsonResponse(['mes_donnees' => $data]);
```